Lecture 2

1. C++ Compilation & Execution Workflow

Before writing algorithmic logic, it is fundamental to understand how C++ processes source code into machine instructions.

- **Source Code:** The high-level instructions written by the programmer in C++.
- **Compilation Phase:** The compiler performs a direct translation (or conversion) of the source code while simultaneously checking for syntax errors.
- **Machine Code Generation:** The compiler outputs an executable binary format consisting entirely of 0s and 1s that the computer's CPU can natively execute.
- **Integrated Development Environment (IDE):** An overarching environment (e.g., VS Code, Code Blocks) that assists programmers in writing, running, and debugging their code.

2. Core Boilerplate & Standard I/O

A standard C++ program requires a specific structural foundation to handle input and output operations properly.

C++

```
#include <iostream>
using namespace std;

int main() {
    cout << "Namaste Duniya" << endl;
    return 0;
}
```

- **`#include <iostream>`:** A preprocessor directive that executes before compilation, importing the standard file necessary for Input/Output operations.
- **`using namespace std;`:** Declares that the program will utilize the standard namespace, preventing the need to prefix standard functions (like `cout`) manually.
- **`int main() { ... }`:** The entry point of the program where execution begins. The curly braces `{}` strictly define the "scope" of the function.
- **`cout <<`:** The command utilized to display output to the standard screen.

- **`endl`:** Instructs the console to terminate the current line and enter a new line, functioning identically to the `\n` escape character or the "Enter" key.

3. Data Types & Memory Allocation

Data types define the nature of the information being stored and dictate the exact amount of hardware memory consumed.

Data Type Comparison Table

Type	Memory Size	Description	Example Initialization
`int`	4 bytes (32 bits)	Stores positive and negative whole numbers.	`int num = 123;`
`unsigned int`	4 bytes (32 bits)	Restricts storage strictly to positive integers, utilizing the maximum bound of $2^{32} - 1$.	`unsigned int a = 10;`
`char`	1 byte (8 bits)	Stores a single ASCII character mapped to an integer value (e.g., `'a'` maps to 97, `'A'` to 65).	`char ch = 'k';`
`bool`	1 byte (8 bits)	Represents a boolean logic state, where 1 denotes True and 0 denotes False (utilizing only 1 functional bit).	`bool bl = true;`
`float`	4 bytes	Stores lower-precision floating-point decimal values.	`float f = 1.2;`
`double`	8 bytes	Stores higher-precision floating-point decimal values.	`double d = 32.678;`

Variable Naming Rules

- **Allowed Characters:** Variable names may strictly contain alphabets, numeric digits, and underscores.
- **Starting Constraints:** A variable name cannot start with a number.
- **Symbol Prohibition:** Variable names cannot contain special symbols such as `%`, `$`, `!`, or `#`.
- **Keyword Restrictions:** You cannot utilize reserved language keywords (like `int`, `cout`, `double`, `bool`) as variable names.
- **Case Sensitivity:** Naming is strictly case-sensitive, and you cannot reuse the exact same variable name within the same scope.

4. Implicit Type Casting

Type casting is the internal mechanism of converting one valid data type into another. When C++ performs this conversion automatically, it is

known as **implicit type casting.**

C++

```
#include <iostream>
using namespace std;

int main() {
    // Character implicitly cast to its ASCII Integer value
    int a = 'a';
    cout << "The value of a is " << a << endl; // Prints 97[cite: 4, 6]

    // Integer implicitly cast to its ASCII Character representation
    char b = 98;
    cout << "The value of b is " << b << endl; // Prints 'b'[cite: 4, 6]

    return 0;
}
```

- **Overflow Edge Case:** If an integer is implicitly cast to a character, but the integer's size is vastly larger than the character's 1-byte limit, the compiler will throw a warning and store only the final byte of the original integer.

5. Bitwise Storage & Retrieval of Negative Numbers

To store signed variables, the Most Significant Bit (MSB) at the absolute front of the binary chain is reserved as the sign indicator: 0 denotes a positive number, and 1 denotes a negative number.

Algorithm for Storing a Negative Number (e.g., -5)

1. Ignore the negative sign and evaluate strictly the positive magnitude of the number (e.g., 5).
2. Generate the binary representation of that positive magnitude padded to the standard bit length ($0000\ldots0101$).
3. Compute the **1's Complement** by flipping every single bit (e.g., $1111\ldots1010$).
4. Compute the **2's Complement** by adding exactly 1 to the generated 1's Complement. The resulting binary string ($1111\ldots1011$) is what is securely saved into memory.

Note: Without the 2's Complement convention, $+0$ and -0 would theoretically have distinct binary representations, wasting memory space.

Algorithm for Displaying a Stored Negative Number

1. The compiler checks the MSB; if it discovers a 1, it recognizes the

underlying number is negative.
2. It retrieves the binary sequence and computes its **1's Complement**.
3. It adds 1 to achieve the **2's Complement**, perfectly reconstructing the absolute positive binary magnitude.
4. The system prints this resulting magnitude with a negative sign explicitly prepended to the standard output.

Note on Unsigned Danger: If you attempt to store a negative integer (like -112) in an `unsigned int` variable, the system applies the exact 2's complement storage steps, but upon printing, it reads all 32 bits straight as a massive positive binary value (printing `4294967184`) instead of taking the display complement.

6. Operators & Expressions

Operators process mathematical and logical transformations on stored variables.

Arithmetic Operators

- **Available Symbols:** Addition (`+`), Subtraction (`-`), Multiplication (`*`), Division (`/`), and Modulo/Remainder (`%`).
- **Division Constraints:** When dividing an integer by an integer (`int/int`), the system forcefully truncates the decimal to return a strict `int` (floor value). For example, $5/2 = 2$.
- **Floating Output:** To retain a decimal answer, at least one parameter must be a float or double. (`int/float = float` , `double/int = double`).

Relational Operators

Evaluate to 1 (True) or 0 (False).

Syntax	Evaluation Rule	Code Check
`==`	Equality Check	`cout << (num1 == num2) << endl;`
`!=`	Not Equal Check	`cout << (num1 != num2) << endl;`
`>` / `<`	Strictly Greater / Strictly Lesser	`cout << (num1 > num2) << endl;`
`>=` / `<=`	Greater or Equal / Lesser or Equal	`cout << (a >= b) << endl;`

Logical Operators

- **Logical AND (`&&`):** Enforces strict compliance; absolutely every conditional check must return True for the final output to be True.

- **Logical OR (`||`):** Highly permissive; at minimum, exactly 1 condition must return True for the final output to evaluate to True.
- **Logical NOT (`!`):** Inverts the binary logic completely, transforming True into False and False into True (Any non-zero parameter automatically evaluates to zero).